

CITY: una piattaforma serverless a microservizi per la gestione intelligente delle emergenze in contesti Smart City

Valerio Torac

Department of Computer Engineering
University of Rome Tor Vergata
Rome, Italy
valerio.torac@alumni.uniroma2.eu

Ali Shalby

Department of Computer Engineering
University of Rome Tor Vergata
Rome, Italy
ali.shalby@alumni.uniroma2.eu

Sommario—Questo articolo presenta CITY, una piattaforma cloud-native e serverless per la gestione intelligente delle emergenze in contesti Smart City. L'obiettivo del progetto è dimostrare come app mobile, microservizi cloud, messaggistica asincrona, analisi immagini e aggiornamenti real-time possano cooperare in una pipeline end-to-end orientata alla sicurezza urbana. CITY riceve segnalazioni manuali da un'app Android, gestisce immagini tramite presigned URL su Amazon S3 e simula eventi generati da una telecamera urbana attraverso AWS IoT Core. Gli eventi vengono analizzati con Amazon Rekognition, disaccoppiati tramite Amazon SQS e orchestrati da AWS Step Functions. Le azioni finali includono notifica tramite Amazon SNS, persistenza in Amazon DynamoDB, archiviazione su Amazon S3 e aggiornamenti WebSocket verso il client mobile. Il progetto è pensato a scopo didattico e prototipale, ma riflette un problema reale: coordinare informazioni eterogenee durante situazioni di emergenza. Dal punto di vista ingegneristico, CITY evidenzia il valore di una architettura modulare, osservabile e riproducibile; dal punto di vista sociale, interpreta la Smart City come un ambiente in cui la tecnologia aiuta a ridurre il tempo tra percezione del pericolo e attivazione della risposta.

Index Terms—Smart City, Serverless, Microservices, AWS Lambda, AWS Step Functions, IoT, Emergency Management, WebSocket.

I. INTRODUZIONE

Una città intelligente non è soltanto un insieme di sensori, API e servizi cloud. È un ecosistema in cui la tecnologia dovrebbe rendere più rapida la comprensione dei problemi e più efficace la risposta verso le persone. In una situazione di emergenza, pochi secondi possono cambiare la percezione del rischio, la qualità dell'intervento e la sicurezza di chi si trova coinvolto. Per questo motivo, un sistema digitale progettato per una Smart City deve saper raccogliere segnali diversi, trasformarli in informazioni affidabili e renderli disponibili in modo chiaro.

CITY nasce da questa visione. Il progetto realizza una piattaforma distribuita per acquisire segnalazioni da cittadini tramite app Android e per simulare eventi generati da una telecamera urbana. Ogni evento viene trattato come una informazione da validare, arricchire, classificare e conservare. L'obiettivo non è sostituire il giudizio umano o i servizi di

emergenza reali, ma costruire un prototipo che mostri come il cloud possa supportare processi critici con tracciabilità, scalabilità e aggiornamenti in tempo reale.

La soluzione è basata su microservizi serverless su Amazon Web Services (AWS). Le funzioni Lambda gestiscono ingestione, analisi, classificazione, decisione, archiviazione e comunicazione. Amazon SQS separa i produttori di eventi dal workflow, AWS Step Functions orchestra l'intera elaborazione e DynamoDB mantiene lo stato persistente. L'app Android non è un semplice form di input: rappresenta il punto di contatto con l'utente, gestisce l'invio delle segnalazioni, l'upload delle immagini e la ricezione degli stati tramite WebSocket.

Dal punto di vista progettuale, CITY è stato concepito come un sistema software completo e non come una demo isolata. Ogni evento possiede un identificativo, ogni fase produce uno stato osservabile e l'infrastruttura è descritta tramite CloudFormation per rendere il deployment riproducibile. Questa impostazione consente di discutere il progetto non solo come applicazione cloud, ma come esempio di integrazione tra mobile computing, event-driven architecture e workflow orchestration.

II. BACKGROUND

A. Serverless computing

Il paradigma serverless permette di eseguire logica applicativa senza gestire direttamente server, sistema operativo o provisioning delle risorse. In AWS Lambda, il codice viene organizzato in funzioni invocate da eventi come richieste HTTP, messaggi in coda, regole IoT o transizioni di un workflow. Questo modello è adatto a un contesto di emergenza perché le richieste possono essere discontinue, improvvise e distribuite nel tempo.

B. Microservizi ed event-driven architecture

CITY divide la logica applicativa in componenti piccoli e specializzati. Alcuni microservizi si occupano dell'ingresso degli eventi, altri dell'analisi e della classificazione, altri ancora della decisione, dell'archiviazione e della comunicazione verso il client. Questa separazione riduce l'accoppiamento e

rende più semplice comprendere, testare e sostituire singole parti della pipeline.

L'architettura è event-driven: API Gateway riceve richieste dall'app, SQS conserva eventi normalizzati, IoT Core gestisce il ramo telecamera e Step Functions coordina l'avanzamento dell'emergenza. La coda SQS è il punto di convergenza dei flussi con immagine e telecamera; in questo modo il workflow non dipende direttamente dalla sorgente che ha generato l'evento.

C. Servizi AWS principali

La piattaforma usa Amazon API Gateway per esporre API HTTP e WebSocket, AWS Lambda per i microservizi, Amazon S3 per immagini, dataset e archivi, Amazon Rekognition per l'analisi visuale, Amazon SQS per la comunicazione persistente, AWS IoT Core per il ramo telecamera, AWS Step Functions per il workflow, Amazon DynamoDB per lo stato condiviso, Amazon SNS per le notifiche e CloudFormation per il deployment.

D. Prospettiva ingegneristica

Il progetto è stato impostato con una prospettiva da ingegneria informatica: non soltanto realizzare una demo funzionante, ma costruire una soluzione manutenibile, osservabile e ridistribuibile. La scelta di separare le responsabilità, usare code persistenti, salvare lo stato e descrivere l'infrastruttura come codice permette di trasformare l'idea iniziale in una architettura controllabile e verificabile.

III. SOLUTION DESIGN

A. Vista generale dell'architettura

La Fig. 1 mostra l'architettura ad alto livello. Il sistema è organizzato attorno a due flussi di ingresso: il flusso manuale da app Android e il flusso telecamera simulato tramite IoT Core. Entrambi convergono verso una pipeline comune che analizza e orchestra l'emergenza.

La convergenza su Amazon SQS è una scelta centrale. L'ingestion può completarsi anche se il workflow non è immediatamente pronto a processare l'evento; successivamente `StartWorkflow` legge dalla coda e avvia la state machine `workflowEmergency`. Questo disaccoppiamento aumenta robustezza e chiarezza architetturale.

B. Applicazione Android

L'app Android è il punto di ingresso del sistema e rende visibile all'utente l'intero percorso dell'emergenza. Dal lato funzionale consente di compilare una segnalazione, allegare una immagine, avviare il test della telecamera simulata e seguire l'avanzamento del workflow. Dal lato architetturale, l'app utilizza quattro endpoint generati dallo stack API: `WebSocketEndpoint`, `UploadUrlEndpoint`, `EmergencyEndpoint` e `CameraTestEndpoint`.

Nel flusso con immagine, l'app non invia la foto direttamente a una Lambda. Prima richiede una presigned URL tramite `POST /upload-url`, poi carica il file su S3 e infine invia la segnalazione con il riferimento all'immagine. Questa scelta

riduce il carico su API Gateway e rende l'upload più adatto a file binari. In parallelo, l'app apre una connessione WebSocket e invia una sottoscrizione basata su `eventId`; in questo modo può ricevere stati progressivi senza polling.

L'importanza dell'app non è soltanto pratica, ma anche comunicativa. In un sistema di emergenza, l'utente non deve percepire il backend come una scatola nera: visualizzare stati come evento validato, classificazione completata, decisione presa e completamento finale aumenta trasparenza e fiducia nel processo.

C. Flusso manuale

Nel flusso manuale l'utente invia una segnalazione dall'app Android. Se non è presente una immagine, `receiveEmergency` può avviare direttamente il workflow. Se invece l'utente allega una foto, `MobileIngestion` recupera l'immagine dal prefix `mobile/` di S3, invoca Amazon Rekognition e costruisce un evento normalizzato da inviare a SQS.

D. Flusso telecamera e IoT

Il secondo ingresso simula una telecamera urbana. L'app invoca `POST /test/camera`; la Lambda `cameraSimulator` seleziona una immagine dal dataset S3 e pubblica un messaggio sul topic `emergency/camera`. Una regola IoT intercetta il messaggio e attiva `lambdaIngestion`, che analizza l'immagine, normalizza l'evento e lo inserisce in SQS. Questo ramo dimostra come una sorgente automatica possa essere integrata nello stesso workflow usato dalle segnalazioni manuali.

E. State machine Step Functions

La Fig. 2 mostra il diagramma del workflow AWS Step Functions. La state machine inizia con la validazione dell'evento e prosegue con contestualizzazione, classificazione, valutazione della gravità e decisione. Tra una fase e l'altra vengono inviati aggiornamenti di stato all'app tramite `SendStatusUpdate`.

La fase `FinalActions` è parallela e contiene tre rami. Il primo gestisce la notifica tramite SNS quando `shouldNotify` è vero. Il secondo salva il record finale in DynamoDB. Il terzo archivia immagine e log tramite `StoreLogs`, gestendo anche il caso di fallimento dell'archiviazione.

F. Persistenza, notifiche e riproducibilità

DynamoDB viene usato per conservare sia i risultati delle emergenze sia le sottoscrizioni WebSocket. La tabella `EmergencyData` mantiene il record finale del workflow, mentre `WebSocketSubscriptions` associa `connectionId` ed `eventId`. Amazon SNS fornisce il canale di notifica verso i soccorritori, ma viene attivato solo quando la decision logic ritiene l'evento rilevante.

L'infrastruttura è divisa in quattro stack CloudFormation: `city-storage-messaging`, `city-lambdas`, `city-api` e `city-workflow-iot`. Gli output degli stack

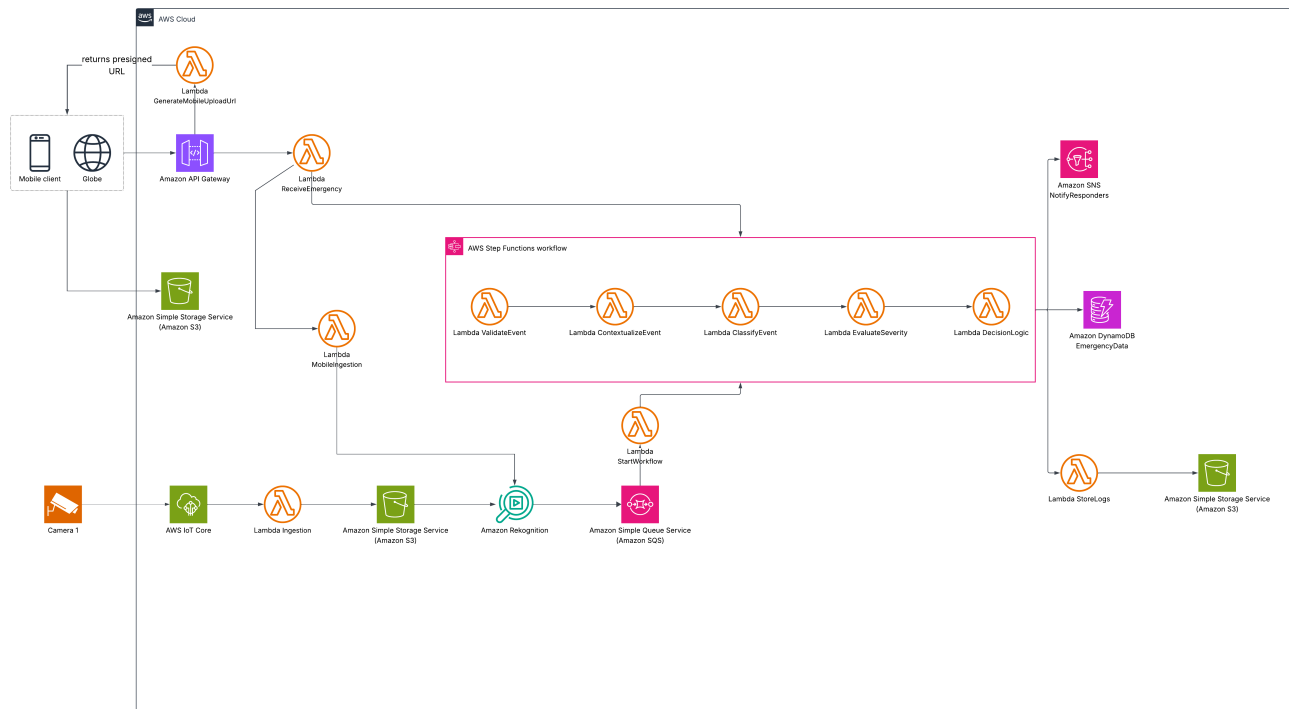


Figura 1. Architettura ad alto livello del sistema CITY. Il diagramma evidenzia il flusso manuale da app mobile, il ramo telecamera/IoT, la coda SQS come punto di convergenza, il workflow Step Functions e le azioni finali su SNS, DynamoDB e S3.

vengono usati per configurare le Lambda e l'app Android, evitando dipendenze implicite da un singolo account AWS.

IV. SOLUTION DETAILS

A. Organizzazione dei microservizi

Il backend è stato organizzato in modo da mantenere una responsabilità chiara per ogni funzione. Le Lambda di ingresso ricevono eventi dall'app o dal ramo IoT; le Lambda di elaborazione preparano l'evento per il workflow; le Lambda interne alla state machine applicano validazione, contestualizzazione, classificazione, valutazione della gravità e decisione; infine, le Lambda di supporto gestiscono aggiornamenti WebSocket e archiviazione.

Questa divisione è importante perché evita di concentrare tutta la logica in un unico componente. In un sistema di emergenza, la chiarezza del percorso è parte della qualità del progetto: se qualcosa fallisce, deve essere possibile capire rapidamente se il problema si trova nell'app, nell'upload dell'immagine, nell'analisi Rekognition, nella coda, nel workflow o nella comunicazione real-time.

B. Endpoint e interazione con il client mobile

L'app Android comunica con il backend attraverso tre endpoint HTTP e un endpoint WebSocket. L'endpoint `/upload-url` viene usato solo quando l'utente allega una immagine; l'endpoint `/emergency` invia la segnalazione manuale; l'endpoint `/test/camera` avvia il ramo di simulazione telecamera. Il WebSocket, invece, rimane aperto per ricevere lo stato del workflow associato allo stesso `eventId`.

Questa separazione rende l'app più pulita: la UI non deve conoscere i dettagli di DynamoDB, SQS, Step Functions o Rekognition. Dal punto di vista dell'utente esistono soltanto azioni comprensibili: segnalare, allegare, avviare un test e seguire l'avanzamento. Il backend traduce queste azioni in una pipeline distribuita.

C. Struttura dei dati e storage

Amazon S3 viene usato con prefix logici differenti. Il prefix `dataset/` contiene le immagini per la telecamera simulata; `mobile/` contiene le immagini caricate dall'app; `captured/` viene usato per il ramo camera; `events/` conserva archivi e log finali. Questa organizzazione rende il bucket unico ma ordinato, facilitando debug e verifica durante i test.

DynamoDB completa la gestione dello stato. La tabella delle emergenze conserva il risultato finale del workflow, mentre la tabella delle sottoscrizioni WebSocket permette di associare una connessione attiva all'evento seguito dall'app. In questo modo il backend può inviare aggiornamenti mirati senza mantenere stato in memoria nelle Lambda.

D. Osservabilità e debugging

Una parte essenziale del progetto è la possibilità di osservare il comportamento del sistema. Step Functions mostra graficamente il percorso dell'evento, CloudWatch raccoglie i log delle Lambda, DynamoDB permette di verificare i record finali e S3 rende ispezionabili le immagini elaborate. Questa osservabilità è utile durante lo sviluppo, ma è anche una caratteristica

necessaria per sistemi critici: un evento importante non deve attraversare il backend senza lasciare tracce comprensibili.

V. RESULTS

A. Validazione dei flussi applicativi

La validazione è stata condotta partendo dai flussi realmente visibili dall'app Android, perché il valore del sistema emerge quando l'utente invia una richiesta e riesce a seguirne l'evoluzione. Il primo scenario ha verificato la segnalazione manuale senza immagine: l'app invia il payload a `/emergency`, il workflow viene avviato e il record finale viene salvato in DynamoDB. In questo caso l'attenzione è posta sulla correttezza del percorso minimo e sulla ricezione degli stati WebSocket fino al completamento.

Il secondo scenario ha introdotto la gestione delle immagini. L'app richiede una presigned URL, carica la foto su S3 nel prefix `mobile/` e invia l'emergenza con il riferimento all'oggetto caricato. Il backend recupera l'immagine, invoca Rekognition, arricchisce l'evento e lo inoltra a SQS. Questo test è rilevante perché verifica la collaborazione tra mobile app, object storage, analisi immagini e workflow asincrono.

Il terzo scenario ha verificato il ramo telecamera. L'app avvia il test, `cameraSimulator` seleziona una immagine dal dataset, IoT Core riceve il messaggio, la regola `CameraIngestionRule` invoca `lambdaIngestion` e l'evento raggiunge la stessa coda SQS usata dal flusso mobile. In questo modo viene validata la possibilità di integrare sorgenti automatiche senza duplicare il workflow applicativo.

B. Comportamento osservato

Durante i test, il sistema ha mostrato una separazione chiara tra acquisizione, elaborazione e comunicazione verso il client. Il caricamento diretto su S3 evita payload HTTP pesanti, SQS protegge il workflow da picchi o ritardi dell'ingestion e Step Functions rende osservabile l'intero percorso dell'emergenza. Ogni stato della state machine espone input, output ed eventuali errori, facilitando debug e dimostrazione.

L'app Android ha avuto un ruolo centrale nella validazione. La ricezione degli stati real-time ha reso il processo comprensibile: l'utente non attende una risposta sincrona opaca, ma vede l'avanzamento dell'elaborazione. Questo aspetto è importante perché un sistema di emergenza deve essere tecnicamente corretto ma anche leggibile da chi lo utilizza.

Un ulteriore risultato riguarda il deployment. Il sistema è stato ricreato su un nuovo account AWS partendo dagli stack CloudFormation e dai pacchetti Lambda caricati su S3. Questo passaggio ha permesso di individuare e correggere dipendenze ambientali, come l'endpoint WebSocket Management API e l'endpoint IoT Data ATS, confermando la riproducibilità del progetto.

Tabella I
SINTESI DEI TEST FUNZIONALI PRINCIPALI.

ID	Test	Esito
T1	Segnalazione manuale senza immagine	Superato
T2	Segnalazione manuale con immagine e upload S3	Superato
T3	Test telecamera con evento IoT	Superato
T4	Analisi immagine con Rekognition	Superato
T5	Avvio workflow da SQS	Superato
T6	Salvataggio su DynamoDB	Superato
T7	Invio aggiornamenti WebSocket all'app	Superato
T8	Deployment CloudFormation su nuovo account	Superato

VI. DISCUSSION

A. Scalabilità, elasticità e fault tolerance

La scalabilità deriva dall'uso di servizi gestiti. API Gateway gestisce le richieste dell'app, Lambda scala in funzione degli eventi, SQS assorbe i picchi, DynamoDB conserva dati persistenti e Step Functions coordina esecuzioni multiple. Questa impostazione è coerente con scenari urbani in cui segnalazioni e sensori possono generare eventi in modo imprevedibile.

La tolleranza ai guasti è ottenuta attraverso il disaccoppiamento. Se l'ingestion produce eventi più velocemente del workflow, SQS li mantiene in coda. Se l'app si disconnette dal WebSocket, il workflow può continuare a salvare dati, inviare notifiche e archiviare log. Gli aggiornamenti real-time migliorano l'esperienza utente, ma non bloccano la logica principale.

B. Significato morale e sociale del progetto

Il significato morale di CITY è legato all'idea che la tecnologia debba essere al servizio delle persone. Una Smart City non è davvero intelligente se raccoglie dati senza trasformarli in cura, attenzione e capacità di risposta. Il progetto propone un modello in cui una segnalazione non viene trattata come un semplice payload tecnico, ma come un evento che merita tracciabilità, analisi e comunicazione.

Questo valore è particolarmente importante nelle emergenze, dove il problema non è soltanto rilevare un evento, ma coordinare fonti diverse e ridurre l'incertezza. Una foto inviata da un cittadino, un messaggio generato da una telecamera e una valutazione automatica non hanno valore se restano isolati. CITY mostra come questi elementi possano convergere in un flusso unico, più leggibile e più utile per supportare una risposta tempestiva.

Allo stesso tempo, il progetto non propone una delega cieca alle decisioni automatiche. Classificazione e severità sono strumenti di supporto, non sostituti del giudizio umano. Una soluzione di questo tipo richiede privacy, sicurezza, supervisione e attenzione ai falsi positivi. Il valore non sta nell'automazione in sé, ma in una automazione progettata per migliorare la sicurezza collettiva senza ridurre dignità, controllo e responsabilità delle persone coinvolte.

C. Limiti

CITY rimane un prototipo e non un sistema certificato per la gestione reale delle emergenze. La telecamera è simulata tramite immagini presenti su S3, la classificazione dipende dalle label restituite da Rekognition e la decision logic è dimostrativa. Queste scelte sono coerenti con l'obiettivo del progetto, ma non esauriscono la complessità di un ambiente urbano reale.

Anche l'app Android, pur essendo funzionale ai flussi principali, non include ancora autenticazione utenti, gestione ruoli o profili differenziati tra cittadino, operatore e amministratore. In uno scenario produttivo sarebbero necessari controlli più forti su identità, autorizzazioni, protezione delle immagini, conservazione dei dati e audit delle decisioni.

D. Sviluppi futuri

Un'evoluzione naturale di CITY sarebbe l'integrazione con sorgenti reali, come telecamere urbane, sensori ambientali o sistemi di monitoraggio già presenti in una città. Questo permetterebbe di passare da una simulazione controllata a un contesto più vicino alla realtà operativa, mantenendo la stessa architettura a eventi.

Un secondo sviluppo riguarda la parte applicativa. L'app Android potrebbe essere estesa con autenticazione, geolocalizzazione più precisa, storico delle segnalazioni, mappe e una comunicazione più ricca verso l'utente. In parallelo, una

dashboard web per operatori permetterebbe di visualizzare eventi, priorità, immagini e stato degli interventi in una vista centrale.

Infine, il progetto potrebbe introdurre metriche operative più avanzate, dashboard CloudWatch, modelli di classificazione più sofisticati, notifiche multi-canale e policy di sicurezza più granulari. In questa direzione, CITY potrebbe diventare una base per sperimentare sistemi urbani più consapevoli, reattivi e orientati al supporto delle persone.

VII. CONCLUSIONI

CITY dimostra come una architettura serverless a micro-servizi possa integrare app mobile, eventi IoT simulati, analisi immagini, code di messaggi, workflow orchestrato, notifiche, database e aggiornamenti real-time. Il contributo principale è la costruzione di una pipeline end-to-end in cui l'evento viene acquisito, arricchito, classificato, valutato, salvato e comunicato all'utente.

Il progetto ha valore tecnico perché mostra una soluzione modulare, scalabile e riproducibile tramite CloudFormation. Ha anche valore umano perché parte da un problema concreto: nelle emergenze, le informazioni arrivano da fonti diverse e devono essere coordinate rapidamente. CITY mostra che il cloud può aiutare a trasformare segnali frammentati in un processo più ordinato, trasparente e utile, mantenendo centrale il ruolo dell'uomo nella valutazione e nell'intervento.